

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO -2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Design Task

Weightage: 20%

QUESTIONS:

Total Marks: 20

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

- (a) Identify key components required in a navigation bar for an e-commerce website.
 - (b) Design a responsive navigation bar using appropriate CSS techniques.
 - (c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).
 - (d) Evaluate the usability and suggest improvements for better user experience.
-

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.

- (a) Identify suitable CSS properties for hover effects.
 - (b) Suggest appropriate **CSS animation or transition** for product interaction.
 - (c) Design a hover effect for a product card.
 - (d) Evaluate performance and user experience considerations.
-

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.

- (a) Identify layout requirements of a dashboard (rows, columns, alignment).

- (b) Compare **Flexbox and Grid** based on layout needs.
 - (c) Choose the most suitable layout technique and justify your choice.
 - (d) Evaluate the scalability and responsiveness of the chosen approach.
-

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.

- (a) Identify key sections of a blog layout.
 - (b) Design a mobile-first layout using CSS.
 - (c) Demonstrate how the layout scales for larger screens using media queries.
 - (d) Evaluate accessibility and readability of the design.
-

Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

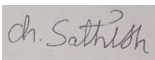
Suppose that a website includes a dynamic menu that responds to user interactions.

- (a) Identify appropriate JavaScript events for menu interaction.
- (b) Design event handling logic for menu toggle functionality.
- (c) Implement interaction for user actions (click/hover).
- (d) Evaluate performance and suggest improvements.

Code of Conduct:

*I **Ch.Sathish / 192311032** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30	Well-structured, efficient, and responsive	Correct implementation with minor issues	Basic implementation with	Incorrect or incomplete implementation

		design implementation		inconsistencies	
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Answers

Question 1: Responsive Navigation Bar Design (4 Marks)

(a) Key components of an e-commerce navigation bar

- Website logo
- Home link
- Product categories
- Search bar
- Cart icon
- User account/Login
- Contact or Help link

(b) Responsive Navigation Bar (HTML & CSS)

HTML

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="style.css">
</head>
<body>

<nav class="navbar">
  <div class="logo">ShopEasy</div>

  <ul class="nav-links" id="menu">
    <li><a href="#">Home</a></li>
    <li><a href="#">Products</a></li>
    <li><a href="#">Categories</a></li>
    <li><a href="#">Cart</a></li>
    <li><a href="#">Login</a></li>
  </ul>

  <div class="menu-icon">&#9776;</div>
</nav>

</body>
</html>
CSS
body{
  margin:0;
  font-family:Arial;
}

.navbar{
```

```
display:flex;
justify-content:space-between;
align-items:center;
background:#333;
padding:15px;
}
```

```
.logo{
  color:white;
  font-size:22px;
}
```

```
.nav-links{
  display:flex;
  list-style:none;
}
```

```
.nav-links li{
  margin-left:20px;
}
```

```
.nav-links a{
  color:white;
  text-decoration:none;
}
```

(c) Mobile Adaptation (Hamburger Menu)

```
.menu-icon{
  display:none;
  color:white;
  font-size:25px;
  cursor:pointer;
}
```

```
@media(max-width:768px){
```

```
.nav-links{
  display:none;
  flex-direction:column;
  width:100%;
}
```

```
.menu-icon{
  display:block;
}
```

```
}
```

(d) Evaluation

- Easy navigation on all devices.
- Hamburger menu saves screen space.
- Search bar can be added for faster product search.
- Larger touch buttons improve mobile usability.

Question 2: Product Hover Effect (4 Marks)

(a) Suitable CSS Properties

- transform
- transition
- box-shadow
- opacity
- background-color

(b) CSS Animation/Transition

Use the transition property for smooth hover effects.

transition:0.3s ease;

(c) Product Card Hover Effect

HTML

```
<div class="card">
  
  <h3>Smart Watch</h3>
  <p>₹2999</p>
</div>
```

CSS

```
.card{
  width:220px;
  padding:15px;
  border:1px solid gray;
  transition:0.3s;
}

.card:hover{
  transform:scale(1.05);
  box-shadow:0 4px 12px gray;
}
```

(d) Evaluation

- Smooth animations improve user engagement.
- Avoid excessive animations for better performance.
- Keep hover effects simple for faster page loading.
- Ensure accessibility for keyboard users.

Question 3: Layout Selection (Flexbox vs Grid) (4 Marks)

(a) Dashboard Layout Requirements

- Multiple rows and columns
- Sidebar
- Header
- Content area
- Proper alignment and spacing

(b) Flexbox vs Grid

Flexbox

One-dimensional layout

Best for rows or columns

Easy alignment

Good for navigation

Grid

Two-dimensional layout

Best for rows and columns

Complex layouts

Good for dashboards

(c) Best Choice

CSS Grid is the best choice because dashboards require both row and column arrangements.

Example:

```
.container{  
  display:grid;  
  grid-template-columns:200px 1fr;  
  grid-template-rows:60px auto;  
  gap:10px;  
}
```

(d) Evaluation

- Easily scalable by adding new grid items.
- Responsive using media queries.
- Better organization of dashboard components.
- Cleaner and easier to maintain.

Question 4: Mobile-First Blog Layout (4 Marks)

(a) Key Sections

- Header
- Navigation
- Blog Content
- Sidebar
- Footer

(b) Mobile-First CSS

```
.container{  
  display:flex;  
  flex-direction:column;  
  padding:10px;  
}
```

```
header,footer{
  background:#333;
  color:white;
  text-align:center;
  padding:15px;
}
```

(c) Media Query for Larger Screens

```
@media(min-width:768px){
```

```
.container{
  flex-direction:row;
}
```

```
.content{
  flex:3;
}
```

```
.sidebar{
  flex:1;
}
}
```

(d) Evaluation

- Mobile-first ensures better mobile performance.
- Larger readable fonts improve readability.
- Responsive layout adapts to different devices.
- High color contrast improves accessibility.

Question 5: JavaScript Event Handling for Dynamic Menu (4 Marks)

(a) JavaScript Events

- click
- mouseover
- mouseout
- keydown

(b) Event Handling Logic

- User clicks the hamburger icon.
- Menu opens or closes.
- Clicking again hides the menu.

(c) JavaScript Implementation

HTML

```
<button id="btn">☰</button>
```

```
<ul id="menu" style="display:none;">
```

```
<li>Home</li>
```



```
<li>Products</li>
<li>Contact</li>
</ul>
JavaScript
let btn = document.getElementById("btn");
let menu = document.getElementById("menu");

btn.addEventListener("click", function(){

    if(menu.style.display=="none"){
        menu.style.display="block";
    }
    else{
        menu.style.display="none";
    }

});
```

(d) Evaluation

- Event listeners provide efficient interaction.
- Avoid unnecessary event handlers for better performance.
- Add smooth CSS transitions for better user experience.
- Support keyboard navigation for improved accessibility.